1) SAME TREE :

```
import java.util.*;

class TreeNode {
   int val;
   TreeNode left, right;

   TreeNode(int val) {
      this.val = val;
   }
}

class Main {

   // Build tree from list
   public static TreeNode buildTree(Integer[] arr, int i) {
      if(i >= arr.length || arr[i] == null)
         return null;

      TreeNode root = new TreeNode(arr[i]);

      root.left = buildTree(arr, 2*i + 1);
      root.right = buildTree(arr, 2*i + 2);

      return root;
   }

   // Same Tree logic
   public static boolean isSameTree(TreeNode p, TreeNode q) {
      if(p == null && q == null) return true;
      if(p == null || q == null) return false;
      if(p.val != q.val) return false;

      return isSameTree(p.left, q.left) &&
           isSameTree(p.right, q.right);
   }

   public static void main(String[] args) {

      // Example with 8+ nodes
      Integer[] p = {1,2,3,4,5,6,7,8};
      Integer[] q = {1,2,3,4,5,6,7,8};

      TreeNode t1 = buildTree(p, 0);
      TreeNode t2 = buildTree(q, 0);

      System.out.println(isSameTree(t1, t2));
   }
}
```

2) SYMMETRIC TREE:

```
import java.util.*;

public class Main{

   public static Tree formtree(Integer[] arr, int i)
   {
      if(i>=arr.length || arr[i]==null)
      {
         return null;
      }

      Tree root = new Tree(arr[i]);

      root.left = formtree(arr,2*i+1);
      root.right = formtree(arr,2*i+2);


      return root;
   }

   public static boolean issymmetric(Tree t1)
   {
      if(t1 == null)
      {
         return true;
      }

      return ismirror(t1.left,t1.right);
   }

   public static boolean ismirror(Tree left, Tree right)
   {
      if(left==null && right==null)
      {
         return true;
      }
      if(left==null || right==null)
      {
         return false;
      }
      if(left.val!=right.val)
      {
         return false;
      }

      return ismirror(left.left,right.right) && ismirror(left.right,right.left);
   }


   public static void main(String[] args)
```

```
    {
        Integer[] p = {1,2,2,null,3,null,3};

        Tree t1 = formtree(p,0);

        System.out.println(issymmetric(t1));
    }
}

class Tree
{
    int val;
    Tree left,right;

    Tree(int x)
    {
        this.val = x;
    }
}
```

3) Max area in island :

```
import java.util.*;

public class Main
{
    public static int dfs(int[][] grid, int i , int j)
    {
        if(i<0 || j<0 || i>=grid.length || j>=grid[0].length || grid[i][j]==0 )
        {
            return 0 ;
        }

        grid[i][j]=0;

        return 1
            +dfs(grid,i+1,j)
            +dfs(grid,i-1,j)
            +dfs(grid,i,j+1)
            +dfs(grid,i,j-1);
    }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        int m= sc.nextInt();
        int n= sc.nextInt();

        int[][] grid = new int[m][n];

        for(int i = 0 ; i < m ; i++)
        {
            for(int j = 0 ; j < n ; j++)
            {
```

```
            grid[i][j]=sc.nextInt();
        }
    }

    int max_area=0;

    for(int i=0 ; i<m ; i++)
    {
        for(int j = 0 ; j < n ; j++)
        {
            if(grid[i][j]==1)
            {
                int area = dfs(grid,i,j);
                max_area= Math.max(max_area,area);
            }

        }
    }

    System.out.println(max_area);

  }

}
```

4) COURSE SCHEDULE :

```
import java.util.*;

class Main {

   public static boolean dfs(int node , List<List<Integer>> graph , int[] state)
   {

      if(state[node]==1)
      {
         return false;
      }
      if(state[node]==2)
      {
         return true;
      }

      state[node]=1;

      for(int nei : graph.get(node))
      {
         if(!dfs(nei,graph,state))
         {
            return false;
         }
      }

      state[node]=2;
      return true;
```

```
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int numcourse = sc.nextInt();
        int n = sc.nextInt();

        List<List<Integer>> graph = new ArrayList<>();


        for(int i=0 ; i<numcourse ; i++)
        {
            graph.add(new ArrayList<>());
        }

        for(int i=0 ; i<n ; i++)
        {
            int a = sc.nextInt();
            int b = sc.nextInt();

            graph.get(a).add(b);
        }

        int[] state = new int[numcourse];

        for(int i=0 ; i<numcourse ; i++)
        {
            if(!dfs(i,graph,state))
            {
                System.out.println(false);
                return;
            }
        }

        System.out.println(true);

    }
}
```

5) Majority Elements :

```
class Solution {
    public int majorityElement(int[] nums) {

        LinkedHashMap<Integer,Integer> map = new LinkedHashMap<>();
```

```
        for(int num : nums)
        {
            map.put(num,map.getOrDefault(num,0)+1);
        }
```

```
        for(int num : map.keySet())
        {
            if(map.get(num)>(nums.length)/2)
            {
                return num;
            }
```

```
        }

        return -1;
    }
}
```

6) Unique path :

```
import java.util.*;

class Main {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      int m = sc.nextInt();
      int n = sc.nextInt();

      int[][] dp = new int[m][n];

      // first row = 1
      for(int j = 0; j < n; j++) {
         dp[0][j] = 1;
      }

      // first column = 1
      for(int i = 0; i < m; i++) {
         dp[i][0] = 1;
      }

      // fill rest
      for(int i = 1; i < m; i++) {
         for(int j = 1; j < n; j++) {
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
         }
      }

      System.out.println(dp[m-1][n-1]);
   }
}
```

7) Climbing stairs :

```
import java.util.*;

class Main {
   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      int n = sc.nextInt();

      if(n == 1) {
```

```
        System.out.println(1);
        return;
      }

      int a = 1, b = 2;

      for(int i = 3; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
      }

      System.out.println(b);
    }
}
```

8) Coin change :

```
import java.util.*;

class Main {

   public static void main(String[] args)
   {

      int[] coins = {1};

      int amount = 0;

      int[] dp = new int[amount+1];

      for(int i=0 ; i<=amount ; i++)
      {
        dp[i]=amount+1;
      }

      dp[0]=0;

      for(int i=1 ; i<=amount ; i++)
      {
        for(int coin : coins)
        {
          if(i-coin >= 0)
          {
            dp[i] = Math.min(dp[i],dp[i-coin]+1);
          }
        }
      }

      if(dp[amount]>amount)
      {
        System.out.println(-1);
        return;
      }

      System.out.println(dp[amount]);
```

```
    }
}
```